

## DAY 1

Problem Statement:

- For pairs of brands in the same year (e.g. apple/samsung/2020 and samsung/apple/2020)
  - if custom1 = custom3 and custom2 = custom4 : then keep only one pair
- For pairs of brands in the same year
  - if custom1 != custom3 OR custom2 != custom4 : then keep both pairs
- For brands that do not have pairs in the same year : keep those rows as well

| brand1  | brand2  | year | custom1 | custom2 | custom3 | custom4 |
|---------|---------|------|---------|---------|---------|---------|
| apple   | samsung | 2020 | 1       | 2       | 1       | 2       |
| samsung | apple   | 2020 | 1       | 2       | 1       | 2       |
| apple   | samsung | 2021 | 1       | 2       | 5       | 3       |
| samsung | apple   | 2021 | 5       | 3       | 1       | 2       |
| google  | NULL    | 2020 | 5       | 9       | NULL    | NULL    |
| oneplus | nothing | 2020 | 5       | 9       | 6       | 3       |

Expected output :

| brand1  | brand2  | year | custom1 | custom2 | custom3 | custom4 |
|---------|---------|------|---------|---------|---------|---------|
| apple   | samsung | 2020 | 1       | 2       | 1       | 2       |
| apple   | samsung | 2021 | 1       | 2       | 5       | 3       |
| samsung | apple   | 2021 | 5       | 3       | 1       | 2       |
| google  | NULL    | 2020 | 5       | 9       | NULL    | NULL    |
| oneplus | nothing | 2020 | 5       | 9       | 6       | 3       |

**Query:**

```
with cte as (select *,
case
when
brand1<brand2 then concat(Brand1,brand2,year)
else concat(Brand2,brand1,year)
end as pair_Id  from brands),
cte_rn as (select *,ROW_NUMBER() over(partition by pair_id order by pair_Id) as rn from cte)
select brand1,brand2,year,custom1,custom2,custom3,custom4 from cte_rn
where rn=1 or (custom1!=custom3 and custom2!=custom4)
```

## DAY 2

Problem statement:

A ski resort company is planning to construct a new ski slope using a pre-existing network of mountain huts and trails between them. A new slope has to begin at one of the mountain huts, have a middle station at another hut connected with the first one by a direct trail, and end at the third mountain hut which is also connected by a direct trail to the second hut. The altitude of the three huts chosen for constructing the ski slope has to be strictly decreasing.

You are given two SQL tables, `mountain_huts` and `trails`, with the following structure:

| id | name     | altitude |
|----|----------|----------|
| 1  | Dakonat  | 1900     |
| 2  | Natasa   | 2100     |
| 3  | Gajantut | 1600     |
| 4  | Rifat    | 782      |
| 5  | Tupur    | 1370     |

| hut1 | hut2 |
|------|------|
| 1    | 3    |
| 3    | 2    |
| 3    | 5    |
| 4    | 5    |
| 1    | 5    |

Assume that:

- There is no trail going from a hut back to itself.
- For every two huts there is at most one direct trail connecting them.
- Each hut from table `trails` occurs in table `mountain_huts`.

Expected result:

| startPt  | middlePt | endPt |
|----------|----------|-------|
| Dakonat  | Gajantut | Tupur |
| Natasa   | Gajantut | Tupur |
| Gajantut | Tupur    | Rifat |
| Dakonat  | Tupur    | Rifat |

**Query:**

```
with cte as (  
    select t1.hut1 as start_hut, h1.name as start_hut_name,  
    h1.altitude as start_hut_Alt, t1.hut2 as end_hut  
    from mountain_huts h1  
    join trails t1 on t1.hut1=h1.id),  
cte2 as (  
    select c1.*,h2.name as end_hut_name, h2.altitude as End_hut_Alt,  
    case when start_hut_Alt>h2.altitude then 1 else 0 end as Altitude_Flag  
    from cte c1  
    join mountain_huts h2 on c1.end_hut=h2.id),  
cte_final as  
(select  
    case when Altitude_flag = 1 then start_hut else end_hut end as start_hut,  
    case when Altitude_flag = 1 then start_hut_name else end_hut_name end as start_hut_name,  
    case when Altitude_flag = 1 then end_hut else Start_hut end as end_hut,  
    case when Altitude_flag = 1 then end_hut_name else Start_hut_name end as end_hut_name  
    from cte2)  
select c1.start_hut_name as startPt,  
c1.end_hut_name as middlePt,  
c2.end_hut_name as endPt from cte_final c1  
join cte_final c2 on c1.end_hut=c2.start_hut
```

## DAY 3

PROBLEM STATEMENT: Write a sql query to return the footer values from input table, meaning all the last non null values from each field as shown in expected output.

| id | car            | length | width | height |
|----|----------------|--------|-------|--------|
| 1  | Hyundai Tucson | 15     | 6     | NULL   |
| 2  | NULL           | NULL   | NULL  | 20     |
| 3  | NULL           | 12     | 8     | 15     |
| 4  | Toyota Rav4    | NULL   | 15    | NULL   |
| 5  | Kia Sportage   | NULL   | NULL  | 18     |

Expected result:

| Car          | length | width | height |
|--------------|--------|-------|--------|
| Kia Sportage | 12     | 15    | 18     |

### **Solution 1:**

```
with cte as (select top 1 car from footer where car is not null order by id desc),
cte2 as (select top 1 length from footer where length is not null order by id desc),
cte3 as (select top 1 width from footer where width is not null order by id desc),
cte4 as (select top 1 height from footer where height is not null order by id desc)
select * from cte,cte2,cte3,cte4
```

### **Solution 2:**

```
select * from
(select top 1 car from footer where car is not null order by id desc) car
cross join (select top 1 length from footer where length is not null order by id desc) length
cross join (select top 1 width from footer where width is not null order by id desc) width
cross join (select top 1 height from footer where height is not null order by id desc) height
```

### **Solution 3:**

with cte as

(select

id,car,length,width,height,

case when car is not null then 1 else 0 end as car\_Flag,

sum(case when car is not null then 1 else 0 end) over (order by id) as car\_segment,

case when length is not null then 1 else 0 end as length\_Flag,

sum(case when length is not null then 1 else 0 end) over (order by id) as length\_segment,

case when width is not null then 1 else 0 end as width\_Flag,

sum(case when width is not null then 1 else 0 end) over (order by id) as width\_segment,

case when height is not null then 1 else 0 end as height\_Flag,

sum(case when height is not null then 1 else 0 end) over (order by id) as height\_segment

from footer),

cte1 as (select top 1 car,FIRST\_VALUE(car) over (Partition by car\_segment order by id) as new\_car  
from cte order by id desc),

cte2 as (select top 1 car,FIRST\_VALUE(length) over (Partition by length\_segment order by id) as  
new\_length from cte order by id desc),

cte3 as (select top 1 car,FIRST\_VALUE(width) over (Partition by width\_segment order by id) as  
new\_width from cte order by id desc),

cte4 as (select top 1 car,FIRST\_VALUE(height) over (Partition by height\_segment order by id) as  
new\_height from cte order by id desc)

select a.new\_car as Car,b.new\_length as length,c.new\_width as width,d.new\_height as height from  
cte1 a, cte2 b,cte3 c,cte4 d

## **DAY 4**

Problem Statement: Derive expected output

Given table

| id | name  | location |
|----|-------|----------|
| 1  | NULL  | NULL     |
| 2  | David | NULL     |
| 3  | NULL  | London   |
| 4  | NULL  | NULL     |
| 5  | David | NULL     |

Expected result:

| id | name  | location |
|----|-------|----------|
| 1  | David | London   |

| id | name  | location |
|----|-------|----------|
| 5  | David | London   |

Solution:

```
select min(id) as id,min(name) as name, min(location) as location from Q4_data
```

```
union all
```

```
select max(id) as id,max(name) as name, max(location) as location from Q4_data
```

## DAY 5

PROBLEM STATEMENT: Using the given Salary, Income and Deduction tables, first write an sql query to populate the Emp\_Transaction table as shown below and then generate a salary report as shown.

| SALARY |          |             |
|--------|----------|-------------|
| EMP_ID | EMP_NAME | BASE_SALARY |
| 1      | Rohan    | 5000        |
| 2      | Alex     | 6000        |
| 3      | Maryam   | 7000        |

| INCOME |           |            |
|--------|-----------|------------|
| ID     | INCOME    | PERCENTAGE |
| 1      | Basic     | 100        |
| 2      | Allowance | 4          |
| 3      | Others    | 6          |

| DEDUCTION |           |            |
|-----------|-----------|------------|
| ID        | DEDUCTION | PERCENTAGE |
| 1         | Insurance | 5          |
| 2         | Health    | 6          |
| 3         | House     | 4          |

| EXPECTED OUTPUT - EMP_TRANSACTION |          |           |        |
|-----------------------------------|----------|-----------|--------|
| EMP_ID                            | EMP_NAME | TRNS_TYPE | AMOUNT |
| 1                                 | Rohan    | Insurance | 250    |
| 2                                 | Alex     | Insurance | 300    |
| 3                                 | Maryam   | Insurance | 350    |
| 1                                 | Rohan    | House     | 200    |
| 2                                 | Alex     | House     | 240    |
| 3                                 | Maryam   | House     | 280    |
| 1                                 | Rohan    | Basic     | 5000   |
| 2                                 | Alex     | Basic     | 6000   |
| 3                                 | Maryam   | Basic     | 7000   |
| 1                                 | Rohan    | Health    | 300    |
| 2                                 | Alex     | Health    | 360    |
| 3                                 | Maryam   | Health    | 420    |
| 1                                 | Rohan    | Allowance | 200    |
| 2                                 | Alex     | Allowance | 240    |
| 3                                 | Maryam   | Allowance | 280    |
| 1                                 | Rohan    | Others    | 300    |
| 2                                 | Alex     | Others    | 360    |
| 3                                 | Maryam   | Others    | 420    |

| EXPECTED OUTPUT - SALARY REPORT |       |           |        |       |           |        |       |                  |         |
|---------------------------------|-------|-----------|--------|-------|-----------|--------|-------|------------------|---------|
| EMPLOYEE                        | BASIC | ALLOWANCE | OTHERS | GROSS | INSURANCE | HEALTH | HOUSE | TOTAL_DEDUCTIONS | NET_PAY |
| Alex                            | 6000  | 240       | 360    | 6600  | 300       | 360    | 240   | 900              | 5700    |
| Maryam                          | 7000  | 280       | 420    | 7700  | 350       | 420    | 280   | 1050             | 6650    |
| Rohan                           | 5000  | 200       | 300    | 5500  | 250       | 300    | 200   | 750              | 4750    |

Solution:

Step1

with cte as

(select income as trns\_type,percentage from income

union

select deduction as trns\_type,percentage from deduction),

cte2 as

(select s.emp\_id as id,s.emp\_name as name,s.base\_salary as salary,cte.\* from cte

cross join salary s)

select cte2.id,cte2.name,cte2.trns\_type,(salary \* (CAST(percentage AS float) / 100)) AS Amount

into emp\_transaction

from cte2

## Step2

```
select name,Allowance,basic,others,
(Allowance+basic+others) as Gross,
Insurance,Health,House,
(Insurance+Health+House) as Total_Deductions,
((Allowance+basic+others)-(Insurance+Health+House)) as net_pay
from
(
select name, trns_type,Amount from emp_transaction
) bq
pivot
(
sum(amount)
for trns_type in ([Allowance],[basic],[others],[Insurance],[Health],[House])
)p
```



## DAY 6

### PROBLEM STATEMENT:

You are given a table having the marks of one student in every test.

You have to output the tests in which the student has improved his performance.

For a student to improve his performance he has to score more than the previous test.

Provide 2 solutions, one including the first test score and second excluding it.

| INPUT   |       |
|---------|-------|
| TEST_ID | MARKS |
| 100     | 55    |
| 101     | 55    |
| 102     | 60    |
| 103     | 58    |
| 104     | 40    |
| 105     | 50    |

| OUTPUT - 1 |       |
|------------|-------|
| TEST_ID    | MARKS |
| 100        | 55    |
| 102        | 60    |
| 105        | 50    |

| OUTPUT - 2 |       |
|------------|-------|
| TEST_ID    | MARKS |
| 102        | 60    |
| 105        | 50    |

#### Solution 1:

```
with cte as (select *,
lag(marks,1,0) over (order by test_id) as com_marks
from student_tests),
cte2 as (select *,
case when (marks>com_marks) then 1
else 0
end as IsGreater from cte)
select test_id,marks from cte2
where Isgreater=1
```

#### Solution 2:

```
with cte as (select *,
lag(marks) over (order by test_id) as com_marks
from student_tests)
select test_id,marks from cte where marks>com_marks
```

## DAY 7

### PROBLEM STATEMENT:

In the given input table DAY\_INDICATOR field indicates the day of the week with the first character being Monday, followed by Tuesday and so on.

Write a query to filter the dates column to showcase only those days where day\_indicator character for that day of the week is 1.

| Product_ID | Day_Indicator | Dates      |
|------------|---------------|------------|
| AP755      | 1010101       | 2024-03-04 |
| AP755      | 1010101       | 2024-03-05 |
| AP755      | 1010101       | 2024-03-06 |
| AP755      | 1010101       | 2024-03-07 |
| AP755      | 1010101       | 2024-03-08 |
| AP755      | 1010101       | 2024-03-09 |
| AP755      | 1010101       | 2024-03-10 |
| XQ802      | 1000110       | 2024-03-04 |
| XQ802      | 1000110       | 2024-03-05 |
| XQ802      | 1000110       | 2024-03-06 |
| XQ802      | 1000110       | 2024-03-07 |
| XQ802      | 1000110       | 2024-03-08 |
| XQ802      | 1000110       | 2024-03-09 |
| XQ802      | 1000110       | 2024-03-10 |

Expected output:

| Product_Id | Day_Indicator | Dates      |
|------------|---------------|------------|
| AP755      | 1010101       | 2024-03-04 |
| AP755      | 1010101       | 2024-03-06 |
| AP755      | 1010101       | 2024-03-08 |
| AP755      | 1010101       | 2024-03-10 |
| XQ802      | 1000110       | 2024-03-04 |
| XQ802      | 1000110       | 2024-03-08 |
| XQ802      | 1000110       | 2024-03-09 |

Solution:

```
with cte as (  
  select *,  
  case when (Cast((datepart(dw,dates)) as int)-1)!=0 then (Cast((datepart(dw,dates)) as int)-1)  
  else (Cast((datepart(dw,dates)) as int)+6)  
  end as dow  
  from Day_Indicator),  
cte2 as  
(select *,  
  case when SUBSTRING(Day_Indicator,dow,1)=1 then 1  
  else 0  
  end as flag  
  from cte)  
select Product_Id,Day_Indicator,Dates from cte2 where flag=1
```

## DAY 8

### PROBLEM STATEMENT:

In the given input table, there are rows with missing JOB\_ROLE values. Write a query to fill in those blank fields with appropriate values.

Assume row\_id is always in sequence and job\_role field is populated only for the first skill.

Provide two different solutions to the problem.

| INPUT  |                |                  |  |  | OUTPUT |                |                  |
|--------|----------------|------------------|--|--|--------|----------------|------------------|
| ROW_ID | JOB_ROLE       | SKILLS           |  |  | ROW_ID | JOB_ROLE       | SKILLS           |
| 1      | Data Engineer  | SQL              |  |  | 1      | Data Engineer  | SQL              |
| 2      |                | Python           |  |  | 2      | Data Engineer  | Python           |
| 3      |                | AWS              |  |  | 3      | Data Engineer  | AWS              |
| 4      |                | Snowflake        |  |  | 4      | Data Engineer  | Snowflake        |
| 5      |                | Apache Spark     |  |  | 5      | Data Engineer  | Apache Spark     |
| 6      | Web Developer  | Java             |  |  | 6      | Web Developer  | Java             |
| 7      |                | HTML             |  |  | 7      | Web Developer  | HTML             |
| 8      |                | CSS              |  |  | 8      | Web Developer  | CSS              |
| 9      | Data Scientist | Python           |  |  | 9      | Data Scientist | Python           |
| 10     |                | Machine Learning |  |  | 10     | Data Scientist | Machine Learning |
| 11     |                | Deep Learning    |  |  | 11     | Data Scientist | Deep Learning    |
| 12     |                | Tableau          |  |  | 12     | Data Scientist | Tableau          |

#### Solution1:

```
select row_id,updatedJobRole as job_role,skills from
(select * ,
FIRST_VALUE(job_role) over (partition by flag2 order by row_id) as updatedJobRole
from
(select * ,
sum(case when job_role is null then 0 else 1 end ) over (order by row_id) as flag2
from job_skills) x)y
```

#### Solution 2:

```
with cte as
(select row_id, job_role, skills from job_skills where row_id=1
union all
select e.row_id, case when e.job_role is null then cte.job_role else e.job_role end as job_role
, e.skills from job_skills e
join cte on e.row_id = cte.row_id + 1)
select * from cte;
```

**DAY 9****PROBLEM STATEMENT:**

Write an sql query to merge products per customer for each day as shown in expected output.

| INPUT       |           |            | OUTPUT    |          |
|-------------|-----------|------------|-----------|----------|
| CUSTOMER_ID | DATES     | PRODUCT_ID | DATES     | PRODUCTS |
| 1           | 2/18/2024 | 101        | 2/18/2024 | 101      |
| 1           | 2/18/2024 | 102        | 2/18/2024 | 101,102  |
| 1           | 2/19/2024 | 101        | 2/18/2024 | 102      |
| 1           | 2/19/2024 | 103        | 2/18/2024 | 104      |
| 2           | 2/18/2024 | 104        | 2/18/2024 | 104,105  |
| 2           | 2/18/2024 | 105        | 2/18/2024 | 105      |
| 2           | 2/19/2024 | 101        | 2/19/2024 | 101      |
| 2           | 2/19/2024 | 106        | 2/19/2024 | 101,103  |
|             |           |            | 2/19/2024 | 101,106  |
|             |           |            | 2/19/2024 | 103      |
|             |           |            | 2/19/2024 | 106      |

**Solution:**

```
select Dates,cast(product_id as varchar) as products from orders
```

```
union
```

```
select dates,STRING_AGG(cast(product_id as varchar),'') as products from orders
```

```
group by customer_id,dates order by dates
```

**DAY 10**

| client | auto | repair date | indicator | value   |
|--------|------|-------------|-----------|---------|
| c1     | a1   | 2022        | level     | good    |
| c1     | a1   | 2022        | velocity  | 90      |
| c1     | a1   | 2023        | level     | regular |
| c1     | a1   | 2023        | velocity  | 80      |
| c1     | a1   | 2024        | level     | wrong   |
| c1     | a1   | 2024        | velocity  | 70      |
| c2     | a1   | 2022        | level     | good    |
| c2     | a1   | 2022        | velocity  | 90      |
| c2     | a1   | 2023        | level     | wrong   |
| c2     | a1   | 2023        | velocity  | 50      |
| c2     | a2   | 2024        | level     | good    |
| c2     | a2   | 2024        | velocity  | 80      |

| velocity | good | wrong | regular |
|----------|------|-------|---------|
| 50       | 0    | 1     | 0       |
| 70       | 0    | 1     | 0       |
| 80       | 1    | 0     | 1       |
| 90       | 2    | 0     | 0       |

Solution:

```
select * from
```

```
(select l.value as level, v.value as velocity from auto_repair l
```

```
join auto_repair v on l.client=v.client and l.auto=v.auto and l.repair_date=v.repair_date
```

```
where l.indicator='level' and v.indicator='velocity' ) q
```

```
pivot(
```

```
count(level)
```

```
for level in ([good],[wrong],[regular])
```

```
) p
```

**DAY 11**

**PROBLEM STATEMENT:** In the given input table, there are hotel ratings which are either too high or too low compared to the standard ratings the hotel receives each year. Write a query to identify and exclude these outlier records as shown in expected output below.

Your output should follow the same order of records as shown.

| hotel            | year | rating |
|------------------|------|--------|
| Radisson Blu     | 2020 | 4.8    |
| Radisson Blu     | 2021 | 3.5    |
| Radisson Blu     | 2022 | 3.2    |
| Radisson Blu     | 2023 | 3.8    |
| InterContinental | 2020 | 4.2    |
| InterContinental | 2021 | 4.5    |
| InterContinental | 2022 | 1.5    |
| InterContinental | 2023 | 3.8    |

Expected output

| Hotel            | Year | Rating |
|------------------|------|--------|
| InterContinental | 2021 | 4.5    |
| InterContinental | 2020 | 4.2    |
| InterContinental | 2023 | 3.8    |
| Radisson Blu     | 2022 | 3.2    |
| Radisson Blu     | 2021 | 3.5    |
| Radisson Blu     | 2023 | 3.8    |

Query 1:

```
with cte as (select hotel, round(Avg(Rating), 2) as avg from hotel_ratings group by hotel),
cte2 as (select h.hotel, h.year, h.rating, c.avg from cte c join hotel_ratings h on h.hotel=c.hotel),
cte3 as (select cte2.*, round(abs(Rating-avg), 2) as difference
from cte2 ),
cte4 as (select *, rank() over (partition by hotel order by difference desc) as rank
from cte3)
select Hotel, Year, Rating from cte4 where rank != 1 order by hotel desc, year
```

Query2:

```
with cte as (select *, round(avg(rating) over(partition by hotel order by year
range between unbounded preceding and unbounded following), 2) as avg_rating
```

```
from hotel_ratings),  
  
cte2 as (select *,round(abs(rating-avg_rating),2) as diff from cte ),  
  
cte3 as (select *,  
            rank() over (partition by hotel order by diff desc) as rank  
from cte2 )  
  
select Hotel,Year,Rating from cte3 where rank !=1 order by hotel desc, year
```

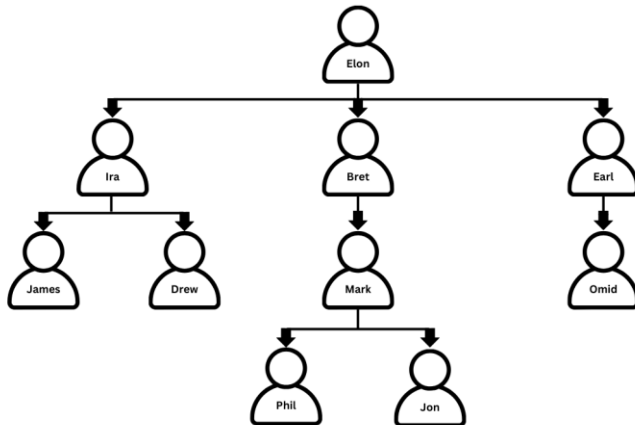


## DAY 12

### PROBLEM STATEMENT:

Given graph shows the hierarchy of employees in a company.

Write an SQL query to split the hierarchy and show the employees corresponding to their team.



### EXPECTED OUTPUT:

| TEAMS  | MEMBERS                     |
|--------|-----------------------------|
| Team 1 | Elon, Bret, Mark, Phil, Jon |
| Team 2 | Elon, Earl, Omid            |
| Team 3 | Elon, Ira, James, Drew      |

Query:

with cte as

```
(select c.employee,c.manager,concat('Team ',ROW_NUMBER() over(order by c.employee)) as Team from
company c
```

```
join company c1 on c1.employee=c.manager where c1.manager is null),
```

rec\_cte as

```
( select employee,manager as CEO, team from cte
```

```
union all
```

```
select c.employee, m.CEO, team from company c
```

```
inner join rec_cte m on c.manager = m.employee )
```

```
select Team, concat(CEO,',',members) as members from
```

```
(select team, String_agg(employee,',') as members,CEO from rec_cte group by team,CEO) x
```

**DAY 13****PROBLEM STATEMENT:**

Find out the no. of employees managed by each manager.

| INPUT |           |         |
|-------|-----------|---------|
| ID    | NAME      | MANAGER |
| 1     | Sundar    |         |
| 2     | Kent      | 1       |
| 3     | Ruth      | 1       |
| 4     | Alison    | 1       |
| 5     | Clay      | 2       |
| 6     | Ana       | 2       |
| 7     | Philipp   | 3       |
| 8     | Prabhakar | 4       |
| 9     | Hiroshi   | 4       |
| 10    | Jeff      | 4       |
| 11    | Thomas    | 1       |
| 12    | John      | 15      |
| 13    | Susan     | 15      |
| 14    | Lorraine  | 15      |
| 15    | Larry     | 1       |

| OUTPUT  |                 |
|---------|-----------------|
| MANAGER | NO_OF_EMPLOYEES |
| Sundar  | 5               |
| Alison  | 3               |
| Larry   | 3               |
| Kent    | 2               |
| Ruth    | 1               |

**Query:**

```
select m.name as managerName,count(e.name) as emp_count from employee_managers e
join employee_managers m on m.id=e.manager group by m.name order by emp_count desc
```

**DAY 14**

**PROBLEM STATEMENT:** In the given input table, some of the invoice are missing, write a sql query to identify the missing serial no.

As an assumption, consider the serial no with the lowest value to be the first generated invoice and the highest serial no value to be the last generated invoice

| INPUT     |              |
|-----------|--------------|
| SERIAL_NO | INVOICE_DATE |
| 330115    | 3/1/2024     |
| 330120    | 3/1/2024     |
| 330121    | 3/1/2024     |
| 330122    | 3/2/2024     |
| 330125    | 3/2/2024     |

| OUTPUT            |
|-------------------|
| MISSING_SERIAL_NO |
| 330116            |
| 330117            |
| 330118            |
| 330119            |
| 330123            |
| 330124            |

**Query:**

WITH MinMax AS (

SELECT MIN(serial\_no) AS min\_s\_no, MAX(serial\_no) AS max\_s\_no FROM invoice

),

Numbers AS (

SELECT min\_s\_no AS s\_no FROM MinMax

UNION ALL

SELECT s\_no + 1 FROM Numbers WHERE s\_no < (SELECT max\_s\_no FROM MinMax)

)

SELECT s\_no FROM Numbers

Except select serial\_no from invoice

**DAY 15****PROBLEM STATEMENT:**

For the given friends, find the no of mutual friends.

| INPUT   |         |
|---------|---------|
| FRIEND1 | FRIEND2 |
| Jason   | Mary    |
| Mike    | Mary    |
| Mike    | Jason   |
| Susan   | Jason   |
| John    | Mary    |
| Susan   | Mary    |

| OUTPUT  |         |               |
|---------|---------|---------------|
| FRIEND1 | FRIEND2 | MUTAL_FRIENDS |
| Jason   | Mary    | 2             |
| John    | Mary    | 0             |
| Mike    | Jason   | 1             |
| Mike    | Mary    | 1             |
| Susan   | Jason   | 1             |
| Susan   | Mary    | 1             |

**Solution 1:**

with all\_friends as(

select friend1,friend2 from Friends

union all

select friend2,friend1 from Friends)

select f.friend1,f.friend2,count(af.friend2) as mutual\_Friends from friends f

left join all\_friends af on f.friend1=af.Friend1

and af.friend2 in (select af2.friend2 from all\_friends af2 where af2.friend1=f.friend2)

group by f.friend1,f.friend2 order by f.friend1,f.friend2

**Solution 2:**

with all\_friends as(

select friend1,friend2 from Friends

union all

select friend2,friend1 from Friends)

select distinct f.\*,count(af.friend2) over (partition by f.friend1,f.friend2 order by f.friend1,f.friend2

range between unbounded preceding and unbounded following ) as mutual\_Friends

from friends f

left join all\_friends af on f.friend1=af.Friend1

and af.friend2 in (select af2.friend2 from all\_friends af2 where af2.friend1=f.friend2)

**DAY 16**

PROBLEM STATEMENT: Given table contains reported covid cases in 2020.

Calculate the percentage increase in covid cases each month versus cumulative cases as of the prior month.

Return the month number, and the percentage increase rounded to one decimal. Order the result by the month.

| INPUT          |           |
|----------------|-----------|
| CASES_REPORTED | DATE      |
| 20124          | 1/10/2020 |
| 40133          | 1/15/2020 |
| 65005          | 1/20/2020 |
| 30005          | 2/8/2020  |
| 35015          | 2/19/2020 |
| 15015          | 3/3/2020  |
| 35035          | 3/10/2020 |
| 49099          | 3/14/2020 |
| 84045          | 3/20/2020 |
| 100106         | 3/31/2020 |
| 17015          | 4/4/2020  |
| 36035          | 4/11/2020 |
| 50099          | 4/13/2020 |
| 87045          | 4/22/2020 |
| 101101         | 4/30/2020 |
| 40015          | 5/1/2020  |
| 54035          | 5/9/2020  |
| 71099          | 5/14/2020 |
| 82045          | 5/21/2020 |
| 90103          | 5/25/2020 |
| 99103          | 5/31/2020 |
| 11015          | 6/3/2020  |
| 28035          | 6/10/2020 |
| 38099          | 6/14/2020 |
| 45045          | 6/20/2020 |
| 36033          | 7/9/2020  |
| 40011          | 7/23/2020 |

| OUTPUT |                     |
|--------|---------------------|
| MONTH  | PERCENTAGE_INCREASE |
| 1      | -                   |
| 2      | 51.9                |
| 3      | 148.9               |
| 4      | 61.5                |
| 5      | 57.1                |
| 6      | 10.2                |
| 7      | 5.7                 |
| 8      | 3.9                 |
| 9      | 7.9                 |
| 10     | 1.7                 |
| 11     | 6.4                 |
| 12     | 7.4                 |

|       |            |
|-------|------------|
| 25001 | 8/12/2020  |
| 29990 | 8/26/2020  |
| 20112 | 9/4/2020   |
| 43991 | 9/18/2020  |
| 51002 | 9/29/2020  |
| 26587 | 10/25/2020 |
| 11000 | 11/7/2020  |
| 35002 | 11/16/2020 |
| 56010 | 11/28/2020 |
| 15099 | 12/2/2020  |
| 38042 | 12/11/2020 |
| 73030 | 12/26/2020 |

**Solution:**

with cte as (

select month(dates) as months,sum(cases\_reported) as no\_cases from covid\_cases group by month(dates)

),

cte2 as

(select \*,cast(sum(no\_cases) over (order by months) as float) as cummulative

from cte)

select months,case

when

months>1 then CAST(round((no\_cases/(lag(cummulative) over (order by months))\*100),2) as VARCHAR)

else '-' end as percentag\_increase from cte2

## DAY 17

### Problem Statement:

User login table shows the date when each user logged in to the system. Identify the users who logged in for 5 or more consecutive days. Return the user id, start date, end date and no of consecutive days.

Please remember a user can login multiple times during a day but only consider users whose consecutive logins spanned 5 days or more.

| USER_ID | LOGIN_DATE | USER_ID | LOGIN_DATE |
|---------|------------|---------|------------|
| 1       | 2024-03-01 | 2       | 2024-03-01 |
| 1       | 2024-03-02 | 2       | 2024-03-02 |
| 1       | 2024-03-03 | 2       | 2024-03-03 |
| 1       | 2024-03-04 | 2       | 2024-03-04 |
| 1       | 2024-03-06 | 3       | 2024-03-01 |
| 1       | 2024-03-10 | 3       | 2024-03-02 |
| 1       | 2024-03-11 | 3       | 2024-03-03 |
| 1       | 2024-03-12 | 3       | 2024-03-04 |
| 1       | 2024-03-13 | 3       | 2024-03-04 |
| 1       | 2024-03-14 | 3       | 2024-03-04 |
| 1       | 2024-03-20 | 3       | 2024-03-05 |
| 1       | 2024-03-25 | 4       | 2024-03-01 |
| 1       | 2024-03-26 | 4       | 2024-03-02 |
| 1       | 2024-03-27 | 4       | 2024-03-03 |
| 1       | 2024-03-28 | 4       | 2024-03-04 |
| 1       | 2024-03-29 | 4       | 2024-03-04 |
| 1       | 2024-03-30 |         |            |

Expected output:

| USER_ID | START_DATE | END_DATE   | CONSECUTIVE_DAYS |
|---------|------------|------------|------------------|
| 1       | 2024-03-10 | 2024-03-14 | 5                |
| 1       | 2024-03-25 | 2024-03-30 | 6                |
| 3       | 2024-03-01 | 2024-03-05 | 5                |

### Query:

```
with cte as ( select *,
dateadd(day,-DENSE_RANK() over(partition by user_id order by user_id,login_date),login_date) as Date_group
from user_login )

select user_id,Date_group, min(login_date) as start_date,max(login_date) as end_Date,DATEDIFF(day,
min(login_date),max(login_date))+1 as consecutive_Days
```



### 30 Days SQL Challenge

```
from cte group by user_id,Date_group having DATEDIFF(day, min(login_date),max(login_date))+1>4  
order by user_id
```

**DAY 18****PROBLEM STATEMENT:**

Find out the employees who attended all the company events.

| EMPLOYEE |         |
|----------|---------|
| ID       | NAME    |
| 1        | Lewis   |
| 2        | Max     |
| 3        | Charles |
| 4        | Sainz   |

| EVENTS         |        |          |
|----------------|--------|----------|
| EVENT_NAME     | EMP_ID | DATES    |
| Product launch | 1      | 3/1/2024 |
| Product launch | 3      | 3/1/2024 |
| Product launch | 4      | 3/1/2024 |
| Conference     | 2      | 3/2/2024 |
| Conference     | 2      | 3/3/2024 |
| Conference     | 3      | 3/2/2024 |
| Conference     | 4      | 3/2/2024 |
| Training       | 3      | 3/4/2024 |
| Training       | 2      | 3/4/2024 |
| Training       | 4      | 3/4/2024 |
| Training       | 4      | 3/5/2024 |

| OUTPUT        |              |
|---------------|--------------|
| EMPLOYEE_NAME | NO_OF_EVENTS |
| Charles       | 3            |
| Sainz         | 3            |

**Solution:**

with cte as (

select distinct id,name,event\_name

from employees e

join events ev on ev.emp\_id=e.id)

select Name,count(1) as no\_Of\_Events from cte group by name

having count(1) in (select Count(Distinct event\_name) from events)

## DAY 19

Given table showcases details of pizza delivery order for the year of 2023. If an order is delayed then the whole order is given for free. Any order that takes more than 30 minutes from the Ordered time is considered as a delayed order. Identify the percentage of delayed order for each month and also display the total no of free pizzas given each month.

Sample Data:

| order_id | order_time              | expected_delivery       | actual_delivery         | no_of_pizzas | price |
|----------|-------------------------|-------------------------|-------------------------|--------------|-------|
| 1        | 2023-09-29 21:22:57.000 | 2023-09-29 21:52:57.000 | 2023-09-29 21:52:57.000 | 8            | 59    |
| 2        | 2023-06-03 19:32:40.000 | 2023-06-03 20:02:40.000 | 2023-06-03 20:02:40.000 | 2            | 83    |
| 3        | 2023-03-19 18:14:04.000 | 2023-03-19 18:44:04.000 | 2023-03-19 18:44:04.000 | 5            | 78    |
| 4        | 2023-06-21 18:45:00.000 | 2023-06-21 19:15:00.000 | 2023-06-21 19:15:00.000 | 3            | 71    |
| 5        | 2023-11-16 20:24:03.000 | 2023-11-16 20:54:03.000 | 2023-11-16 20:54:03.000 | 1            | 73    |
| 6        | 2023-06-01 00:17:54.000 | 2023-06-01 00:47:54.000 | 2023-06-01 00:47:54.000 | 9            | 62    |
| 7        | 2023-04-15 15:29:57.000 | 2023-04-15 15:59:57.000 | 2023-04-15 15:59:57.000 | 8            | 60    |
| 8        | 2023-11-03 12:35:06.000 | 2023-11-03 13:05:06.000 | 2023-11-03 13:05:06.000 | 5            | 96    |
| 9        | 2023-01-20 21:36:56.000 | 2023-01-20 22:06:56.000 | 2023-01-20 22:06:56.000 | 6            | 52    |
| 10       | 2023-07-31 13:43:39.000 | 2023-07-31 14:13:39.000 | 2023-07-31 14:13:39.000 | 10           | 82    |

Expected output:

| order_month | percentage | free_pizzas |
|-------------|------------|-------------|
| Jan-2023    | 9.23%      | 31          |
| Feb-2023    | 12.22%     | 49          |
| Mar-2023    | 15.79%     | 61          |
| Apr-2023    | 13.41%     | 77          |
| May-2023    | 14.29%     | 65          |
| Jun-2023    | 10.96%     | 48          |
| Jul-2023    | 15.71%     | 43          |
| Aug-2023    | 11.24%     | 63          |
| Sep-2023    | 18.92%     | 89          |
| Oct-2023    | 15.91%     | 60          |
| Nov-2023    | 23.08%     | 105         |
| Dec-2023    | 15.15%     | 58          |

Query:

with cte as

(

select order\_id,order\_time,month(order\_time) as mm,YEAR(order\_time)as yyyy,expected\_delivery,actual\_delivery,no\_of\_pizzas,price,

datediff(MINUTE,order\_time,actual\_delivery) as delay\_minutes

from pizza\_delivery ) ,

cte2 as

(

select \*, case

when Delay\_minutes >30 then 1 else 0 end as delay\_flag ,

case when Delay\_minutes >30 then no\_of\_pizzas else 0 end as free\_pizzas

from cte),

cte3 as

(

select mm,yyyy,sum(Delay\_Flag) as delayed\_orders,count(1) as flag\_count,

round(((cast(sum(Delay\_Flag) as float)/cast(count(1) as float))\*100,2) as percentage ,

sum(free\_pizzas) as free\_pizzas

from cte2 where actual\_delivery is not null

group by mm,yyyy

)

select

concat(substring(DATENAME(MONTH, DATEADD(MONTH, mm - 1, '1900-01-01')),1,3),'-',yyyy) as order\_month,

concat(Cast(Percentage as varchar),'%') as percentage,free\_pizzas

from cte3 order by mm

## DAY 20

PROBLEM STATEMENT: Find the median ages of countries

| INPUT |          |     |
|-------|----------|-----|
| ID    | COUNTRY  | AGE |
| 1     | Poland   | 10  |
| 2     | Poland   | 5   |
| 3     | Poland   | 34  |
| 4     | Poland   | 56  |
| 5     | Poland   | 45  |
| 6     | Poland   | 60  |
| 7     | India    | 18  |
| 8     | India    | 15  |
| 9     | India    | 33  |
| 10    | India    | 38  |
| 11    | India    | 40  |
| 12    | India    | 50  |
| 13    | USA      | 20  |
| 14    | USA      | 23  |
| 15    | USA      | 32  |
| 16    | USA      | 54  |
| 17    | USA      | 55  |
| 18    | Japan    | 65  |
| 19    | Japan    | 6   |
| 20    | Japan    | 58  |
| 21    | Germany  | 54  |
| 22    | Germany  | 6   |
| 23    | Malaysia | 44  |

| OUTPUT   |     |
|----------|-----|
| COUNTRY  | AGE |
| Germany  | 6   |
| Germany  | 54  |
| India    | 33  |
| India    | 38  |
| Japan    | 58  |
| Malaysia | 44  |
| Poland   | 34  |
| Poland   | 45  |
| USA      | 32  |

Query:

```
select country, age from
```

```
(select *,
```

```
ROW_NUMBER() over (partition by country order by age) as rk,
```

Cast(count(1) over (partition by country order by age

range between unbounded preceding and unbounded following) as float) as count\_Per\_Country from people) x

where (rnk >= count\_Per\_Country/2) and rnk <=(count\_Per\_Country/2+1)

## DAY 21

PROBLEM STATEMENT: The column 'perc\_viewed' in the table 'post\_views' denotes the percentage of the session duration time the user spent viewing a post. Using it, calculate the total time that each post was viewed by users. Output post ID and the total viewing time in seconds, but only for posts with a total viewing time of over 5 seconds.

Sample data:

| session_id | user_id | session_starttime           | session_endtime             | platform | session_id | post_id | perc_viewed |
|------------|---------|-----------------------------|-----------------------------|----------|------------|---------|-------------|
| 1          | U1      | 2020-01-01 12:14:28.0000000 | 2020-01-01 12:16:08.0000000 | Windows  | 1          | 1       | 2           |
| 2          | U1      | 2020-01-01 18:23:50.0000000 | 2020-01-01 18:24:00.0000000 | Windows  | 1          | 2       | 4           |
| 3          | U1      | 2020-01-01 08:15:00.0000000 | 2020-01-01 08:20:00.0000000 | IPhone   | 1          | 3       | 1           |
| 4          | U2      | 2020-01-01 10:53:10.0000000 | 2020-01-01 10:53:30.0000000 | IPhone   | 2          | 1       | 20          |
| 5          | U2      | 2020-01-01 18:25:14.0000000 | 2020-01-01 18:27:53.0000000 | IPhone   | 2          | 2       | 10          |

Expected output:

| post_id | timeviewed |
|---------|------------|
| 4       | 5.1        |
| 2       | 24         |

Query:

with cte as

(select p.\*,s.session\_starttime,s.session\_endtime,

DATEDIFF(SECOND,session\_starttime,session\_endtime) as sessiontime

from user\_sessions s

join post\_views p on p.session\_id=s.session\_id),

cte2 as (select \*,(sessiontime)\*perc\_viewed/100 as totaltime

from cte)

select post\_id, sum(totaltime) as timeviewed from cte2 group by post\_id having sum(totaltime) > 5 order by 2

## DAY 22

Problem Statement: IPL Winning Streak

Given table has details of every IPL 2023 matches. Identify the maximum winning streak for each team.

Additional test cases:

- 1) Update the dataset such that when Chennai Super Kings win match no 17, your query shows the updated streak.
- 2) Update the dataset such that Royal Challengers Bangalore loose all match and your query should populate the winning streak as 0.

Sample Data:

| match_no | round_number | dates      | location                                           | home_team                   | away_team             | result                      |
|----------|--------------|------------|----------------------------------------------------|-----------------------------|-----------------------|-----------------------------|
| 1        | 1            | 2023-03-31 | Narendra Modi Stadium, Ahmedabad                   | Gujarat Titans              | Chennai Super Kings   | Gujarat Titans              |
| 2        | 1            | 2023-04-01 | Punjab Cricket Association IS Bindra Stadium, Moha | Punjab Kings                | Kolkata Knight Riders | Punjab Kings                |
| 3        | 1            | 2023-04-01 | Bharat Ratna Shri Atal Bihari Vajpayee Ekana Crick | Lucknow Super Giants        | Delhi Capitals        | Lucknow Super Giants        |
| 4        | 1            | 2023-04-02 | Rajiv Gandhi International Stadium, Hyderabad      | Sunrisers Hyderabad         | Rajasthan Royals      | Rajasthan Royals            |
| 5        | 1            | 2023-04-02 | M Chinnaswamy Stadium, Bengaluru                   | Royal Challengers Bangalore | Mumbai Indians        | Royal Challengers Bangalore |

Expected output:

| teams                       | max_winning_streak |
|-----------------------------|--------------------|
| Chennai Super Kings         | 3                  |
| Gujarat Titans              | 3                  |
| Lucknow Super Giants        | 3                  |
| Mumbai Indians              | 3                  |
| Rajasthan Royals            | 3                  |
| Royal Challengers Bangalore | 2                  |
| Sunrisers Hyderabad         | 2                  |
| Punjab Kings                | 2                  |
| Kolkata Knight Riders       | 2                  |
| Delhi Capitals              | 2                  |



Query:

```
with cte_teams (teams) as
(select home_team from ipl_results
union
select away_team from ipl_results),
cte as (
select Dates,home_team,away_team,teams,result,
row_number() over (partition by teams order by teams,dates) as rn
from cte_teams t
join ipl_results r on r.home_team=t.teams or r.away_team=t.teams),
cte_3 as (
select *,
rn-row_number() over (partition by teams order by teams,dates) as diff
from cte where result=teams ),
cte_final as
(select teams,
count(1) over (partition by teams,diff order by teams) as streak
from cte_3)
select t.teams,
case when max(streak) is not null then max(streak) else 0 end as max_winning_streak
from cte_teams t
left join cte_final cf on cf.teams=t.teams group by t.teams order by 2 desc
```

**DAY 23**[2153. The Number of Passengers in Each Bus II](#)Table: **Buses**

| +-----+      |      |  |
|--------------|------|--|
| Column Name  | Type |  |
| +-----+      |      |  |
| bus_id       | int  |  |
| arrival_time | int  |  |
| capacity     | int  |  |
| +-----+      |      |  |

bus\_id contains unique values.

Each row of this table contains information about the arrival time of a bus at the LeetCode station and its capacity (the number of empty seats it has).

No two buses will arrive at the same time and all bus capacities will be positive integers.

Table: **Passengers**

| +-----+      |      |  |
|--------------|------|--|
| Column Name  | Type |  |
| +-----+      |      |  |
| passenger_id | int  |  |
| arrival_time | int  |  |
| +-----+      |      |  |

passenger\_id contains unique values.

Each row of this table contains information about the arrival time of a passenger at the LeetCode station.

Buses and passengers arrive at the LeetCode station. If a bus arrives at the station at a time **tbus** and a passenger arrived at a time **tpassenger** where **tpassenger <= tbus** and the passenger did not catch any bus, the passenger will use that bus. In addition, each bus has a capacity. If at the moment the bus arrives at the station there are more passengers waiting than its capacity **capacity**, only **capacity** passengers will use the bus.

Write a solution to report the number of users that used each bus.

Return the result table ordered by `bus_id` in **ascending order**.

The result format is in the following example.

### Example 1:

#### Input:

Buses table:

| bus_id | arrival_time | capacity |
|--------|--------------|----------|
| 1      | 2            | 1        |
| 2      | 4            | 10       |
| 3      | 7            | 2        |

Passengers table:

| passenger_id | arrival_time |
|--------------|--------------|
| 11           | 1            |
| 12           | 1            |
| 13           | 5            |
| 14           | 6            |
| 15           | 7            |

#### Output:

| bus_id | passengers_cnt |
|--------|----------------|
| 1      | 1              |
| 2      | 1              |
| 3      | 2              |

**Explanation:**

- Passenger 11 arrives at time 1.
- Passenger 12 arrives at time 1.
- Bus 1 arrives at time 2 and collects passenger 11 as it has one empty seat.
- Bus 2 arrives at time 4 and collects passenger 12 as it has ten empty seats.
- Passenger 12 arrives at time 5.
- Passenger 13 arrives at time 6.
- Passenger 14 arrives at time 7.
- Bus 3 arrives at time 7 and collects passengers 12 and 13 as it has two empty seats.

**SOLUTION:**

```

with cte_data as (
select ROW_NUMBER()over(order by arrival_time) as rn ,bus_id,capacity,
(select Count(1) from Passengers p where p.arrival_time<=b.arrival_time) as total_passengers
from buses b),
cte as
(
select rn,bus_id,capacity,total_passengers,
case when capacity < total_passengers then capacity else total_passengers end as onboarded_passengers,
case when capacity < total_passengers then capacity else total_passengers end as Total_onboarded_passengers
from cte_data where rn=1
union all
select d.rn,d.bus_id,d.capacity,d.total_passengers,

```

```
case when d.capacity < (d.total_passengers-c.Total_onboarded_passengers)
then d.capacity else (d.total_passengers-c.Total_onboarded_passengers)
end as onboarded_passengers,
case when c.Total_onboarded_passengers+d.capacity < (d.total_passengers-c.Total_onboarded_passengers)
then d.capacity else (d.total_passengers-c.Total_onboarded_passengers)
end as onboarded_passengers
from cte c
join cte_data d on d.rn=c.rn+1
)
select bus_id,onboarded_passengers from cte order by 1
```

## DAY 24

Problem Statement: Find valid email id's

A consumer electronics store in Warsaw stores all the customer feedback in the feedback table. The email ids mentioned by customers are then used by the store to contact customers to promote any upcoming sales. However, some of the customers, while sharing feedback, enter invalid email addresses. Write an SQL query to identify and return all the valid email addresses from the feedback table.

A valid email address needs to have 3 parts:

Part 1 is the username. A username can contain upper or lower-case letters, numbers and special characters like underscore character "\_", dot ".", hyphen "-". Username should always start with a letter.

Part 2 is the "@" symbol.

Part 3 is the domain which needs to have 2 sub parts. The first part contains upper or lower-case letters followed by a dot symbol and then followed by 2 or 3 letters.

Sample data:

| feedback_id | cust_name | email                | rating | remarks   |
|-------------|-----------|----------------------|--------|-----------|
| 1           | Zohan     | zohan@2024@gmail.com | 4      | good      |
| 2           | Keyaan    | keyaan.TR@gmail.com  | 5      | very good |
| 3           | Zayn      | ZAYN...@gmail        | 3      | ok        |
| 4           | Emir      | emir-#1@outlook.com  | 4      | ok        |
| 5           | Ashar     | Ashar-@hotmail.DE    | 4      | nice      |
| 6           | Zoya      | zoya@techTFQ.org     | 4      | great     |
| 7           | Kabir     | kabir.com@-gmail.com | 2      | bad       |
| 8           | Ayaan     | ayaan123@company.net | 1      | poor      |
| 9           | Meir      | meir123@             | 1.5    | poor      |
| 10          | Noah      | noah_.com@.com       | 3.5    | bad       |
| 11          | Idris     | i@gmail.com          | 5      | excellent |
| 12          | Arhaan    | arhaan\$gmail.com    | 5      | awesome   |
| 13          | Abrar     | abrar123@gmail.comm  | 5      | awesome   |

Expected output:

| feedback_id | cust_name | email                | rating | remarks   |
|-------------|-----------|----------------------|--------|-----------|
| 2           | Keyaan    | keyaan.TR@gmail.com  | 5      | very good |
| 5           | Ashar     | Ashar-@hotmail.DE    | 4      | nice      |
| 6           | Zoya      | zoya@techTFQ.org     | 4      | great     |
| 8           | Ayaan     | ayaan123@company.net | 1      | poor      |
| 11          | Idris     | i@gmail.com          | 5      | excellent |

Query:

```
select * from feedback
```

```
where email like '[a-zA-Z]@[a-zA-Z].[a-zA-Z]'
```

```
and email not like '%[^a-zA-Z0-9_-.]@[a-zA-Z].[a-zA-Z]%' and charindex('.',REVERSE(email)) in (4,3)
```

## DAY 25

### PROBLEM STATEMENT:

Analyse the given input table and come up with output as shown.

Sample data:

| store_id | product_1              | product_2              |
|----------|------------------------|------------------------|
| 1        | Apple - iPhone         | Apple - MacBook Pro    |
| 1        | Apple - AirPods        | Samsung - Galaxy Phone |
| 2        | Apple iPhone           | Apple: Phone           |
| 2        | Google Pixel           | apple: Laptop          |
| 2        | Sony: Camera           | Apple Vision Pro       |
| 3        | samsung - Galaxy Phone | mapple MacBook Pro     |

Expected output:

| store_id | product_1 | product_2 |
|----------|-----------|-----------|
| 1        | 2         | 1         |
| 2        | 1         | 3         |
| 3        | 0         | 0         |

Query:

```
select store_id,  
  
sum(case when product_1 like 'Apple%' then 1 else 0 end) as product_1,  
  
sum(case when product_2 like 'Apple%' or product_2 like ' Apple%' or product_2 like ' Apple%' then 1 else 0 end) as  
product_2  
  
from product_demo group by store_id
```



## DAY 26

PROBLEM STATEMENT: Given table contains tokens taken by different customers in a tax office.

Write a SQL query to return the lowest token number which is unique to a customer (meaning token should be allocated to just a single customer).

Sample data:

| token_num | customer |
|-----------|----------|
| 1         | Maryam   |
| 2         | Rocky    |
| 3         | John     |
| 3         | John     |
| 2         | Arya     |
| 1         | Pascal   |
| 9         | Kate     |
| 9         | Ibrahim  |
| 8         | Lilly    |
| 8         | Lilly    |
| 5         | Shane    |

Expected output:

| minimum_Token_num |
|-------------------|
| 3                 |

Solution:

with cte as

(select Distinct \* from tokens),

cte2 as

(select token\_num,count(customer) as count from cte group by token\_num)

select Min(token\_num) as minimum\_Token\_num from cte2 where count=1

## DAY 27

### PROBLEM STATEMENT:

Given vacation\_plans tables shows the vacations applied by each employee during the year 2024. Leave\_balance table has the available leaves for each employee. Write an SQL query to determine if the vacations applied by each employee can be approved or not based on the available leave balance. If an employee has enough available leaves then mention the status as "Approved" else mention "Insufficient Leave Balance". Assume there are no public holidays during 2024. weekends (sat & sun) should be excluded while calculating vacation days.

Sample data:

| id | emp_id | from_dt    | to_dt      |
|----|--------|------------|------------|
| 1  | 1      | 2024-02-12 | 2024-02-16 |
| 2  | 2      | 2024-02-20 | 2024-02-29 |
| 3  | 3      | 2024-03-01 | 2024-03-31 |
| 4  | 1      | 2024-04-11 | 2024-04-23 |
| 5  | 4      | 2024-06-01 | 2024-06-30 |
| 6  | 3      | 2024-07-05 | 2024-07-15 |
| 7  | 3      | 2024-08-28 | 2024-09-15 |

| emp_id | balance |
|--------|---------|
| 1      | 12      |
| 2      | 10      |
| 3      | 26      |
| 4      | 20      |
| 5      | 14      |

Expected output:

| id | emp_id | applied_Vac_days | Status               |
|----|--------|------------------|----------------------|
| 1  | 1      | 5                | Approved             |
| 2  | 2      | 8                | Approved             |
| 3  | 3      | 21               | Approved             |
| 5  | 4      | 20               | Approved             |
| 4  | 1      | 9                | Insufficient balance |
| 6  | 3      | 7                | Insufficient balance |
| 7  | 3      | 13               | Insufficient balance |

Query:

```

with DateSeries as ( select id,emp_id,CAST(from_dt as datetime) as DateValue, to_dt
                    from vacation_plans
                    union all
                    select id,emp_id,DATEADD(day, 1, DateValue), to_dt  from DateSeries  where DateValue < to_dt),
cte as
(select * from DateSeries),
cte2 as
(select *,case when DateName(WEEKDAY,DateValue) in ('Saturday','Sunday') then 0 else 1 end  as weekday from cte),
cte3 as (select
id,c2.emp_id, count(weekday) as applied_Vac_days,l.balance,
ROW_NUMBER() over (partition by c2.emp_id order by c2.emp_id,c2.id) as rn
from cte2 c2
join leave_balance l on l.emp_id=c2.emp_id where weekday<>0 group by id,c2.emp_id,l.balance),
cte4 as (
    select id,emp_id,applied_Vac_days,balance,rn,(balance-applied_vac_days) as remaining_balance
from cte3 where rn=1
union all
select c.id,c.emp_id,c.applied_Vac_days,c.balance,c.rn,(cte4.remaining_balance-c.applied_vac_days) as
remaining_balance
from cte4
join cte3 c on c.rn=cte4.rn+1 and c.emp_id=cte4.emp_id
)
select id,emp_id,applied_Vac_days,
case when remaining_balance>=0 then 'Approved' else 'Insufficient balance' end as Status
from cte4 order by Status,id,emp_id
OPTION (MAXRECURSION 0);

```

## DAY 28

Find length of comma seperated values in items field.

Sample data

| id | items         |
|----|---------------|
| 1  | 221,221,022   |
| 2  | ,6,0,9999     |
| 3  | 100,2000,2    |
| 4  | 4,44,444,4444 |

Expected output

| id | lengths |
|----|---------|
| 1  | 2,3,4   |
| 2  | 0,1,1,4 |
| 3  | 3,4,1   |
| 4  | 1,2,3,4 |

Query:

```
with cte as(
select id,value from item

cross apply string_split(items',')),
cte2 as (select id ,LEN(value) as lengths from cte)
select id,STRING_AGG(lengths',' ) as lengths from cte2 group by id
```

## DAY 29

PROBLEM STATEMENT: Given table provides login and logoff details of one user.

Generate a report to represent the different periods (in mins) when user was logged in.

| INPUT    |        |
|----------|--------|
| TIMES    | STATUS |
| 10:00:00 | on     |
| 10:01:00 | on     |
| 10:02:00 | on     |
| 10:03:00 | off    |
| 10:04:00 | on     |
| 10:05:00 | on     |
| 10:06:00 | off    |
| 10:07:00 | off    |
| 10:08:00 | off    |
| 10:09:00 | on     |
| 10:10:00 | on     |
| 10:11:00 | on     |
| 10:12:00 | on     |
| 10:13:00 | off    |
| 10:14:00 | off    |
| 10:15:00 | on     |
| 10:16:00 | off    |
| 10:17:00 | off    |

| OUTPUT   |          |          |
|----------|----------|----------|
| LOG_ON   | LOG_OFF  | DURATION |
| 10:00:00 | 10:03:00 | 3        |
| 10:04:00 | 10:06:00 | 2        |
| 10:09:00 | 10:13:00 | 4        |
| 10:15:00 | 10:16:00 | 1        |

### Query:

with cte as

(

select \*, ROW\_NUMBER() over (order by times) as rn2

from

(select \*, ROW\_NUMBER() over (order by times) as rn

from login\_details) x where status='on'),

cte2 as

(

select \*,FIRST\_VALUE(times) over (partition by grouping order by grouping,times) as log\_on\_Time,

LAST\_VALUE(times) over (partition by grouping order by grouping,times

range between unbounded preceding and unbounded following) as last\_log\_on\_Time

from

(select \*,rn-rn2 as grouping from cte) y),

cte3 as (

select Distinct log\_on\_Time,Last\_Log\_On\_Time from cte2),

cte4 as

(select \*, lead(ld.times) over (order by ld.times) as log\_Off\_Time

from cte3 c right join login\_details ld on ld.times=c.last\_log\_on\_Time)

select \*, DATEDIFF(minute,log\_On\_Time,Log\_off\_time) as duration\_mins

from

(select Log\_on\_time,log\_off\_time from cte4 where log\_on\_Time is not null) a

## DAY 30

PROBLEM STATEMENT: Given tables represent the marks scored by engineering students.

Create a report to display the following results for each student.

- Student\_id, Student name, Total Percentage of all marks

- Failed subjects (must be comma separated values in case of multiple failed subjects)

- Result (if percentage  $\geq 70\%$  then 'First Class', if  $\geq 50\%$  &  $\leq 70\%$  then 'Second class', if  $\leq 50\%$  then 'Third class' else 'Fail'. The result should be Fail if a students fails in any subject irrespective of the percentage marks)

\*\*\* The sequence of subjects in student\_marks table match with the sequential id from subjects table.

\*\*\* Students have the option to choose either 4 or 5 subjects only.

Sample data:

| roll_no   | name   |
|-----------|--------|
| 2GR5CS011 | Maryam |
| 2GR5CS012 | Rose   |
| 2GR5CS013 | Alice  |
| 2GR5CS014 | Lilly  |
| 2GR5CS015 | Anna   |
| 2GR5CS016 | Zoya   |

| student_id | subject1 | subject2 | subject3 | subject4 | subject5 | subject6 |
|------------|----------|----------|----------|----------|----------|----------|
| 2GR5CS011  | 75       | NULL     | 56       | 69       | 82       | NULL     |
| 2GR5CS012  | 57       | 46       | 32       | 30       | NULL     | NULL     |
| 2GR5CS013  | 40       | 52       | 56       | NULL     | 31       | 40       |
| 2GR5CS014  | 65       | 73       | NULL     | 81       | 33       | 41       |
| 2GR5CS015  | 98       | NULL     | 94       | NULL     | 90       | 20       |
| 2GR5CS016  | NULL     | 98       | 98       | 81       | 84       | 89       |

| id | name                        | pass_marks |
|----|-----------------------------|------------|
| S1 | Mathematics                 | 40         |
| S2 | Algorithms                  | 35         |
| S3 | Computer Networks           | 35         |
| S4 | Data Structure              | 40         |
| S5 | Artificial Intelligence     | 30         |
| S6 | Object Oriented Programming | 35         |

Expected output:

| Student_id | name   | percentage_marks | Failed_subjects                   | Status       |
|------------|--------|------------------|-----------------------------------|--------------|
| 2GR5CS011  | Maryam | 70.5             | -                                 | First class  |
| 2GR5CS012  | Rose   | 41.25            | Data Structure, Computer Networks | Failed       |
| 2GR5CS013  | Alice  | 43.8             | -                                 | Third class  |
| 2GR5CS014  | Lilly  | 58.6             | -                                 | Second class |
| 2GR5CS015  | Anna   | 75.5             | Object Oriented Programming       | Failed       |
| 2GR5CS016  | Zoya   | 90               | -                                 | First class  |

**Solution:**

```

with cte as(

SELECT * FROM student_marks

UNPIVOT (

    marks FOR subject IN (subject1, subject2, subject3, subject4, subject5, subject6)

) AS unpivoted_data),

student as

(select student_id,name,subject,marks from cte c

join students s on s.roll_no=c.student_id),

subject as (select COLUMN_NAME as subjectId,name,pass_marks from

(select COLUMN_NAME,ROW_NUMBER() over(order by ordinal_position) as rn

from INFORMATION_SCHEMA.COLUMNS where TABLE_NAME='Student_marks' and COLUMN_NAME like 'subject%')x

Join (select ROW_NUMBER() over(order by id) as rn,name,pass_marks from subjects) y

on y.rn=x.rn),

percentage as (

select student_id,s.name,su.name as subjectName, marks,pass_marks

--,avg(CAST(marks as float)) as Percentage,

,case when marks>=pass_marks then null else su.name end as failed_subjects

from student s join subject su on su.subjectId=s.subject ),

cte_final as (

select Student_id,name,avg(CAST(marks as float)) as Percentage_marks,string_agg(failed_subjects,', ' ) as

failed_subjects

from percentage group by student_id,name )

select Student_id,name,percentage_marks,

case when failed_subjects is null then '-'

else failed_subjects end as Failed_subjects,

case when failed_subjects is not null then 'Failed'

when percentage_marks>=70 then 'First class'

when percentage_marks between 50 and 70 then 'Second class'

when percentage_marks<50 then 'Third class'          end as Status

from cte_final

```